Generative AI applications for data engineering

Scenario

You are a data engineer who has been hired by a European online retail company to design a data workflow for their operations. The company requires that you perform all of the following tasks:

1. Propose a detailed data architecture for the whole data process. The client does not want cloud-based processing resources. The company wants an SQL-based central data repository that their employees from multiple countries can access for their use.

2. Propose a detailed data warehouse schema and design its entity requirements document (ERD). a. The client wants customer information, seller information, inventory information, and transaction invoice information, to be recorded. b. The client wants the final data prepared such that the final record of sales invoices contains the headers InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

3. Recommend the infrastructure requirements for the proposed data architecture.

4. Create an ETL pipeline to clean, process, and load the data to an SQL server for analysis. Test the pipeline on a sample database. a. The recorded data is available at a provided URL. b. The InvoiceNo starting with the character C is a credit entry, and you should remove these entries from the records before starting your analysis. c. The StockCode values of C2, D, M, and POST correspond to Carraige, Discount, Manual, and Postage entries, none of which are required for analysis. d. The CustomerID is missing from a few entries. Remove these entries from the data before analysis. e. Load the final transaction record to an SQLite3 database Invoice_Records under the table Purchase_transactions.

5. Query the SQL database to access data from the server. You are required to extract the data of a selected country from the database table created in the previous step.

6. Implement data analysis and data mining strategies on the final data. Then, implement Apriori algorithm and perform association rule mining on the data for the specified country.

Credits: Author(s) Abhishek Gagneja

Estimate competition time for this project

Estimating the time to complete this project can vary depending on several factors, such as the complexity of the data, the tools and technologies used, and the expertise of the data engineer. Here's a rough breakdown:

1. **Propose a detailed data architecture for the whole data process**: 8-12 hours
2. **Propose a detailed data warehouse schema and design its entity relationship diagram (ERD)**: 12-16 hours
3. **Propose the infrastructure requirements for the required data architecture**: 6-8 hours
4. **Create an ETL pipeline to clean, process, and load the data to an SQL server for analysis. Test the pipeline on a sample database**: 16-24 hours
5. **Query the SQL database to access data from the server**: 4-6 hours
6. **Implement data analysis and data mining strategies on the final data**: 12-16 hours

In total, the project might take approximately 58-82 hours.

Task 1 Data Architecture

To propose a data architecture for the retail system, you will set up the following guidelines based on the client's specifications.

The client does not want to go for cloud based processing resources. They want an SQL-based central data repository that their employees from multiple countries can access for their use.

You can use the GenAI model to propose a data architecture.

Prompt:

> Create a data architecture, clearly defining each step of the process, for an online retail company.
>
> Please take note of the following specifications.
>
> - The company does not want to use cloud platforms for storage or processing.
>
> - They want an SQL-based central data repository that can be accessed by any employee as per requirements.

Sample response:

1. Define data sources: Identify all the sources of data within the company, such as sales transactions, customer information, inventory data, and so on.

2. Data collection: Set up processes to collect data from the various sources and store it in the on-premises SQL server. This can involve setting up data pipelines to automate the extraction and loading of data into the central repository.

3. Data transformation: Clean and transform the data to ensure consistency and quality. This may involve data cleansing, normalization, and standardization to make the data usable for analysis.

4. Data modeling: Design the database schema and data model to organize the data in a structured and efficient manner. Define relationships between different data entities to facilitate data retrieval and analysis.

5. Data access: Implement access controls and permissions to regulate who can access and modify the data in the central repository. This ensures data security and compliance with privacy regulations.

6. Data analysis: Provide tools and interfaces for employees to access and analyze the data stored in the central repository. This can include dashboards, reporting tools, and data visualization techniques to make data insights easily accessible.

7. Data backup and recovery: Establish regular data backup procedures to prevent data loss in case of system failures or disasters. Implement disaster recovery plans to ensure quick recovery of data in case of emergencies.

8. Data governance: Implement policies and procedures to govern the use and management of data within the company. This includes data quality standards, data retention policies, and data lifecycle management practices.

By following these steps, you can create a comprehensive data architecture for the online retail company that meets their specifications of not using cloud platforms and having an SQL-based central data repository accessible to all employees.

Watson answered prompt:

here is a proposed data architecture for an online retail company that meets the specified requirements:

Data Architecture Overview

The data architecture will consist of the following components:

1. **Data Warehouse**
2. **Data Store**
3. **Data Access Layer**
4. **Data Security and Access Control**
5. **Data Quality and Governance**

Data Warehouse

- **Purpose**: To store and manage large amounts of data in a centralized location, providing a single source of truth for all business data.
- **Components**:
  - **Data Store**: A relational database management system (RDBMS) that stores data in a structured and normalized manner.
  - **Data Catalog**: A metadata repository that provides information about the data, including data definitions, data relationships, and data quality metrics.

    - **Data Quality Engine**: A process that ensures data accuracy, completeness, and consistency.

Data Store

- **Purpose**: To store data in a relational database management system (RDBMS) that is optimized for large-scale data storage and retrieval.
- **Components**:
    - **Database**: A relational database management system (RDBMS) such as MySQL, PostgreSQL, or Oracle.
    - **Data Storage**: A database schema that defines the structure of the data, including tables, columns, and relationships between tables.
    - **Data Schema**: A database schema that defines the data structure and relationships between tables.

Data Access Layer

- **Purpose**: To provide a layer of abstraction between the data store and the data access layer, allowing for secure and efficient data retrieval and manipulation.
- **Components**:
    - **Data Access Service**: A web-based interface that provides a simple and intuitive way for employees to access the data store.
    - **Data Access API**: A RESTful API that provides a standardized interface for data access and manipulation.

    - **Data Security**: A layer that ensures data access is secure and authenticated.

Data Security and Access Control

- **Purpose**: To ensure that only authorized employees can access the data store and data access layer.
- **Components**:
    - **Authentication**: A system that verifies user identity and authorizes access to the data store and data access layer.
    - **Authorization**: A system that controls access to specific data and data access methods.
    - **Data Encryption**: A layer that encrypts data in transit and at rest to protect against unauthorized access.

Data Quality and Governance

- **Purpose**: To ensure the accuracy, completeness, and consistency of the data in the data store.
- **Components**:
    - **Data Quality Engine**: A process that ensures data accuracy, completeness, and consistency.
    - **Data Validation**: A system that validates data against predefined rules and constraints.
    - **Data Auditing**: A system that tracks changes to the data and provides audit trails.

Data Flow

1. **Data Ingestion**: Data is ingested from various sources, such as databases, files, and APIs.
2. **Data Processing**: Data is processed and transformed into a standardized format for storage in the data store.
3. **Data Storage**: Data is stored in the data store.
4. **Data Access**: Data is accessed through the data access layer.
5. **Data Retrieval**: Data is retrieved from the data store and processed for analysis or reporting.
6. **Data Analysis**: Data is analyzed and insights are generated.
7. **Data Reporting**: Data is presented to stakeholders through reports and dashboards.

Data Access Layer

1. **GET /data**: Returns a list of all data in the data store.
2. **GET /data/{id}**: Returns a single record of data with the specified ID.
3. **POST /data**: Creates a new record of data.
4. **PUT /data/{id}**: Updates an existing record of data.
5. **DELETE /data/{id}**: Deletes a record of data.

Data Security

1. **Authentication**: Verifies user identity and authorizes access to the data store.
2. **Authorization**: Controls access to specific data and data access methods.

3. **Data Encryption**: Encrypts data in transit and at rest to protect against unauthorized access.

Data Quality

1. **Data Validation**: Validates data against predefined rules and constraints.
2. **Data Auditing**: Tracks changes to the data and provides audit trails.

This data architecture provides a centralized and secure way to store and manage data for the online retail company, while meeting the specified requirements.

5.10 seconds

Task 2 Data warehouse schema and ERD

First, you need to set up the data warehouse schema and its ERD diagram. For that task, you need clearly defined requirements from the client as to the kind of data they want recorded. Assume that you received the following information from the client.

1. The client wants to record customer information, seller information, inventory information, and transaction invoice information.
2. The client wants the final data prepared such that the final record of sales invoices displayes the headers InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID and Country.

To define the schema of the data warehouse that meets these requirements, you can now write a prompt on the GenAI platform.

Prompt:

Propose a data schema for an online retail store that meets the following guidelines.

Records Customer information, inventory information, and transaction invoice information.

Prepare the final data such that the final record of sales invoices has the headers `InvoiceNo`, `StockCode`, `Description`, `Quantity`, `InvoiceDate`, `UnitPrice`, `CustomerID` and `Country`.

Sample GTP response: To create a data schema for an online retail store that includes customer information, seller information, inventory information, and transaction invoice information, you can design a relational database with the following tables:

1. Customer table:

- CustomerID (Primary key)
- CustomerName
- Address
- Phone
- Email

2. Inventory table:

- StockCode (Primary key)
- Description
- QuantityAvailable
- Price

3. Transaction Invoice table:

- InvoiceNo (Primary key)
- StockCode (Foreign key referencing Inventory table)
- Quantity
- InvoiceDate
- UnitPrice
- CustomerID (Foreign key referencing Customer table)

- Country

By setting up this data schema, you can store and manage customer information, inventory details, and transaction invoice data in a structured and organized manner. The final record of sales invoices can be retrieved by querying the Transaction Invoice table with the specified headers: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

You can further use the generative AI platform to generate the SQL codes for creating this warehouse, and use this code to create the ERD on the DbDiagram interface.

Prompt:

Generate the SQL code to create this data warehouse

Sample response:

Here is the SQL code to create the data warehouse with the specified data schema for an online retail store:

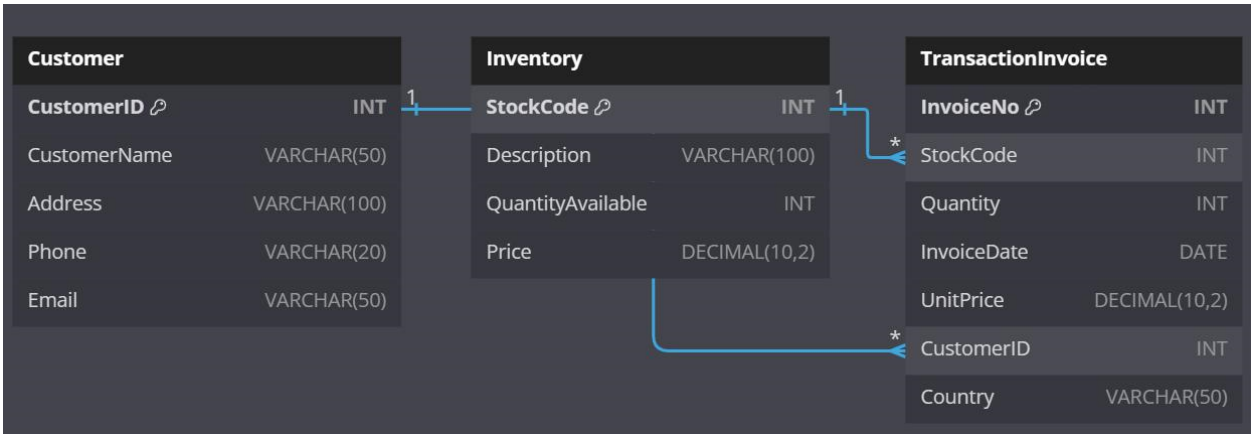
```
1. -- Create Customer Table
2. CREATE TABLE Customer (
3.     CustomerID INT PRIMARY KEY,
4.     CustomerName VARCHAR(50),
5.     Address VARCHAR(100),
6.     Phone VARCHAR(20),
7.     Email VARCHAR(50)
```

```
);

-- Create Inventory Table
CREATE TABLE Inventory (
    StockCode INT PRIMARY KEY,
    Description VARCHAR(100),
    QuantityAvailable INT,
    Price DECIMAL(10, 2)
);

-- Create Transaction Invoice Table
CREATE TABLE TransactionInvoice (
    InvoiceNo INT PRIMARY KEY,
    StockCode INT,
    Quantity INT,
    InvoiceDate DATE,
    UnitPrice DECIMAL(10, 2),
    CustomerID INT,
```

```
26.     Country VARCHAR(50),

27.     FOREIGN KEY (StockCode) REFERENCES Inventory(StockCode),

28.     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)

29. );
```

You can use this SQL code to create the necessary tables for storing customer information, seller information, inventory details, and transaction invoice data in the specified data schema.

sample ERD



Watson answer prompt:

Here is a proposed data schema for the online retail store:

Customer Table

| Column Name | Data Type | Description | | --- | --- | --- | | CustomerID | int | Unique identifier for the customer | | Name | varchar(255) | Customer name | | Email | varchar(255) | Customer email | |

Phone | varchar(20) | Customer phone number | | Address | varchar(255) | Customer address | | Country | varchar(50) | Customer country |

Inventory Table

| Column Name | Data Type | Description | | --- | --- | --- | | InventoryID | int | Unique identifier for the inventory item | | ProductID | int | Foreign key referencing the Product table | | StockLevel | int | Current stock level of the product | | UnitPrice | decimal(10, 2) | Current price of the product | | UnitWeight | int | Weight of the product | | UnitMeasure | varchar(10) | Unit of measurement (e.g. kg, lbs) |

Product Table

| Column Name | Data Type | Description | | --- | --- | --- | | ProductID | int | Unique identifier for the product | | ProductName | varchar(255) | Product name | | Description | text | Product description | | UnitPrice | decimal(10, 2) | Current price of the product | | UnitWeight | int | Weight of the product | | UnitMeasure | varchar(10) | Unit of measurement (e.g. kg, lbs) |

Transaction Table

| Column Name | Data Type | Description | | --- | --- | --- | | TransactionID | int | Unique identifier for the transaction | | CustomerID | int | Foreign key referencing the Customer table | | ProductID | int | Foreign key referencing the Product table | | Quantity | int | Quantity of the product sold | | UnitPrice | decimal(10, 2) | Unit price of the product | | Total | decimal(10, 2) | Total cost of the transaction | | InvoiceDate | date | Date of the transaction | | InvoiceNo | varchar(20) | Invoice number |

Invoice Table

| Column Name | Data Type | Description | | --- | --- | --- | | InvoiceID | int | Unique identifier for the invoice | | CustomerID | int | Foreign key referencing the Customer table | | InvoiceDate | date | Date of the invoice | | InvoiceNo | varchar(20) | Invoice number | | Total | decimal(10, 2) | Total cost of the invoice |

Relationships

- A customer can have many transactions (one-to-many).
- A product can be sold in many transactions (one-to-many).
- An invoice is associated with one customer and one product (many-to-one).
- An invoice is associated with one invoice (many-to-one).

Data Schema

sql

```sql
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(255) NOT NULL,
  Email VARCHAR(255) NOT NULL,
  Phone VARCHAR(20) NOT NULL,
  Address VARCHAR(255) NOT NULL,
  Country VARCHAR(50) NOT NULL
);
```

```
9

10CREATE TABLE Products (

11  ProductID INT PRIMARY KEY,

12  ProductName VARCHAR(255) NOT NULL,

13  Description TEXT NOT NULL,

14  UnitPrice DECIMAL(10, 2) NOT NULL,

15  UnitWeight INT NOT NULL,

16  UnitMeasure VARCHAR(10) NOT NULL

17);

18

19CREATE TABLE Inventory (

20  InventoryID INT PRIMARY KEY,

21  ProductID INT NOT NULL,

22  StockLevel INT NOT NULL,

23  UnitPrice DECIMAL(10, 2) NOT NULL,

24  UnitWeight INT NOT NULL,

25  UnitMeasure VARCHAR(10) NOT NULL,

26  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
```

```
27);

28

29CREATE TABLE Transactions (

30  TransactionID INT PRIMARY KEY,

31  CustomerID INT NOT NULL,

32  ProductID INT NOT NULL,

33  Quantity INT NOT NULL,

34  UnitPrice DECIMAL(10, 2) NOT NULL,

35  Total DECIMAL(10, 2) NOT NULL,

36  InvoiceDate DATE NOT NULL,

37  InvoiceNo VARCHAR(20) NOT NULL,

38  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),

39  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)

40);

41

42CREATE TABLE Invoices (

43  InvoiceID INT PRIMARY KEY,

44  CustomerID INT NOT NULL,
```

45 InvoiceDate DATE NOT NULL,

46 InvoiceNo VARCHAR(20) NOT NULL,

47 Total DECIMAL(10, 2) NOT NULL,

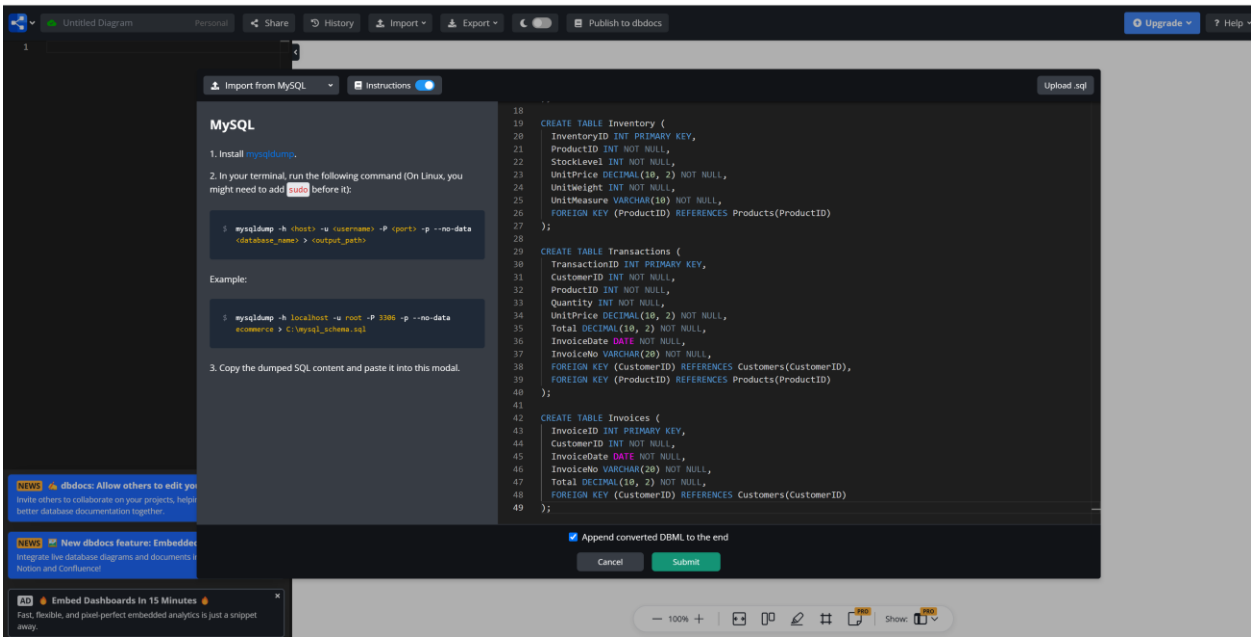
48 FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)

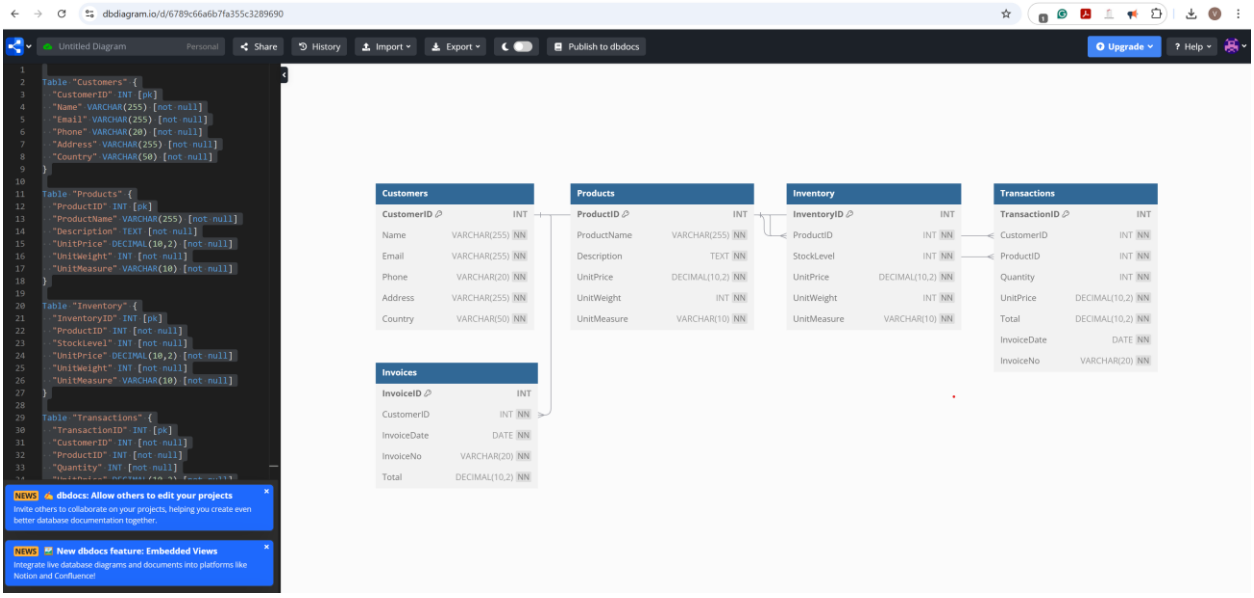
49);

This schema defines the necessary tables and relationships to store customer information, inventory information, and transaction data. The Invoice table is used to store the final record of sales invoices, which includes the

headers InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

6.02 seconds




dbdiagram.io/d/6789c66a6b7fa355c3289690
Untitled Diagram
Personal
Share
History
Import
Export
Publish to dbdocs
Upgrade
Help
Table "Customers" {
"CustomerID" INT [pk]
"Name" VARCHAR(255) [not null]
"Email" VARCHAR(255) [not null]
"Phone" VARCHAR(20) [not null]
"Address" VARCHAR(255) [not null]
"Country" VARCHAR(50) [not null]
}
Table "Products" {
"ProductID" INT [pk]
"ProductName" VARCHAR(255) [not null]
"Description" TEXT [not null]
"UnitPrice" DECIMAL(10,2) [not null]
"UnitWeight" INT [not null]
"UnitMeasure" VARCHAR(10) [not null]
}
Table "Inventory" {
"InventoryID" INT [pk]
"ProductID" INT [not null]
"StockLevel" INT [not null]
"UnitPrice" DECIMAL(10,2) [not null]
"UnitWeight" INT [not null]
"UnitMeasure" VARCHAR(10) [not null]
}
Table "Transactions" {
"TransactionID" INT [pk]
"CustomerID" INT [not null]
"ProductID" INT [not null]
"Quantity" INT [not null]
NEW! dbdocs: Allow others to edit your projects
Invite others to collaborate on your projects, helping you create even better database documentation together.
NEW! New dbdocs feature: Embedded Views
Integrate live database diagrams and documents into platforms like Notion and Confluence!
Customers
CustomerID INT
Name VARCHAR(255) NN
Email VARCHAR(255) NN
Phone VARCHAR(20) NN
Address VARCHAR(255) NN
Country VARCHAR(50) NN
Products
ProductID INT
ProductName VARCHAR(255) NN
Description TEXT NN
UnitPrice DECIMAL(10,2) NN
UnitWeight INT NN
UnitMeasure VARCHAR(10) NN
Inventory
InventoryID INT
ProductID INT NN
StockLevel INT NN
UnitPrice DECIMAL(10,2) NN
UnitWeight INT NN
UnitMeasure VARCHAR(10) NN
Transactions
TransactionID INT
CustomerID INT NN
ProductID INT NN
Quantity INT NN
UnitPrice DECIMAL(10,2) NN
Total DECIMAL(10,2) NN
InvoiceDate DATE NN
InvoiceNo VARCHAR(20) NN
Invoices
InvoiceID INT
CustomerID INT NN
InvoiceDate DATE NN
InvoiceNo VARCHAR(20) NN
Total DECIMAL(10,2) NN

Task 3 Infrastructure requirements

You now need to define the infrastructure requirements for such a setup. You can make use of the same chat for making this prompt as the GenAI will draw context from the previous responses and give you a tailored response.

You will frame a prompt that can use the context from the previous steps and create the infrastructure requirements for your design.

Prompt:

> Propose a set of infrastructure requirements for different stages of the data workflow based on the data architecture discussed here.

Sample response:

Based on the data architecture discussed for the online retail company, the infrastructure requirements for different stages of the data workflow can be outlined as follows:

1. Data collection:
    - Hardware: Servers or devices to collect data from various sources and transfer the data to the central SQL server.
    - Software: Data collection tools and scripts to automate the extraction and load the data into the central repository.
    - Networking: Reliable network connections to ensure the seamless data transfer between sources and the central repository.
2. Data transformation:

- Hardware: Sufficient processing power and memory to handle data transformation tasks efficiently.
- Software: Data transformation tools and ETL (Extract, Transform, Load) processes to clean, normalize, and standardize the data.
- Networking: Fast and stable network connections to support data transformation processes.

3. Data modeling:
    - Hardware: Hardware resources to support database schema design and data modeling activities.
    - Software: Database management software to define relationships between data entities and optimize data storage.
    - Networking: Network connectivity to access and manage the database schema.
4. Data access:
    - Hardware: Devices and systems to access the central data repository for analysis and reporting.
    - Software: Data visualization tools, reporting software, and query interfaces for employees to access and analyze data.
    - Networking: Secure network connections to ensure data access controls and permissions are enforced.
5. Data backup and recovery:

- Hardware: Backup storage devices or systems to store copies of data for disaster recovery purposes.
- Software: Backup and recovery software to automate data backup processes and ensure data integrity.
- Networking: Network connections to facilitate data backup and recovery operations.

6. Data security:
    - Hardware: Security appliances or devices to protect the data warehouse from unauthorized access.
    - Software: Encryption software, access control mechanisms, and data security tools to secure sensitive data.
    - Networking: Secure network configurations, firewalls, and monitoring tools to detect and prevent security breaches.

By addressing these infrastructure requirements at each stage of the data workflow, the online retail company can ensure the efficiency, security, and reliability of their data architecture and operations.

Watson answer:

Here is a proposed set of infrastructure requirements for different stages of the data workflow based on the data architecture discussed:

Data Ingestion

- **Data Ingestion System**: A scalable, high-performance system that can handle large volumes of data from various sources, such as databases, files, and APIs.
- **Data Ingestion Protocol**: A standardized protocol for ingesting data from various sources, such as CSV, JSON, or XML files.
- **Data Ingestion Tools**: Tools for ingesting data from various sources, such as Apache NiFi, Apache Beam, or AWS Glue.
- **Data Ingestion Frequency**: Data ingestion should be performed at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.

Data Processing

- **Data Processing Engine**: A scalable, high-performance engine that can process data in real-time or near-real-time, depending on the requirements.
- **Data Processing Tools**: Tools for processing data, such as Apache Spark, Apache Flink, or Apache Beam.
- **Data Processing Frequency**: Data processing should be performed at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.
- **Data Quality Control**: A system for ensuring data quality, such as data validation, data cleansing, and data transformation.

Data Storage

- **Data Storage System**: A scalable, high-performance storage system that can handle large volumes of data, such as relational databases, NoSQL databases, or object storage.

- **Data Storage Protocols**: Protocols for storing data, such as relational database management systems (RDBMS), NoSQL databases, or object storage systems.
- **Data Storage Frequency**: Data should be stored at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.
- **Data Backup and Recovery**: A system for backing up and recovering data, such as using snapshots or replication.

Data Access

- **Data Access System**: A scalable, high-performance system that provides secure and efficient access to data, such as a web-based interface or a data gateway.
- **Data Access Protocols**: Protocols for accessing data, such as RESTful APIs, GraphQL, or message queues.
- **Data Access Frequency**: Data should be accessed at regular intervals, such as daily, weekly, or monthly, depending on the requirements.
- **Data Security**: A system for securing data access, such as authentication, authorization, and encryption.

Data Quality

- **Data Quality Tools**: Tools for ensuring data quality, such as data validation, data cleansing, and data transformation.
- **Data Quality Frequency**: Data should be checked and validated at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.

- **Data Quality Metrics**: Metrics for measuring data quality, such as data accuracy, completeness, and consistency.

Data Governance

- **Data Governance Framework**: A framework for managing data governance, such as data classification, data mapping, and data lineage.
- **Data Governance Protocols**: Protocols for managing data governance, such as data cataloging, data lineage, and data provenance.
- **Data Governance Frequency**: Data should be governed at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.

Monitoring and Maintenance

- **Monitoring Tools**: Tools for monitoring data workflow, such as monitoring tools for data ingestion, processing, and storage.
- **Monitoring Frequency**: Data should be monitored at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.
- **Maintenance Tools**: Tools for maintaining data workflow, such as data backup and recovery, data archiving, and data migration.

Security

- **Security Protocols**: Protocols for securing data, such as encryption, access controls, and authentication.

- **Security Frequency**: Data should be secured at regular intervals, such as daily, weekly, or monthly, depending on the volume of data.
- **Security Audits**: Regular security audits should be performed to ensure data security.

Scalability

- **Scalable Infrastructure**: Infrastructure that can scale to handle large volumes of data, such as cloud-based infrastructure or on-premises infrastructure.
- **Scalable Storage**: Storage that can scale to handle large volumes of data, such as object storage or cloud-based storage.
- **Scalable Processing**: Processing that can scale to handle large volumes of data, such as cloud-based processing or on-premises processing.

Reliability

- **Reliable Infrastructure**: Infrastructure that can ensure data availability, such as cloud-based infrastructure or on-premises infrastructure.
- **Reliable Storage**: Storage that can ensure data availability, such as object storage or cloud-based storage.
- **Reliable Processing**: Processing that can ensure data availability, such as cloud-based processing or on-premises processing.

Cost-Effective

- **Cost-Effective Infrastructure**: Infrastructure that is cost-effective, such as cloud-based infrastructure or on-premises infrastructure. 6.63 seconds

Task 4 The ETL workflow

At this point, you can assume that the central data collection has taken place and the invoice details are available as a CSV file on remote server. You need to create an ETL pipeline that reads this file, cleans the data based on the client specifications and loads the data to a SQL server. Following specifications for this task have been shared.

1. The recorded data is available on the following URL:

   https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMSkillsNetwork-AI0273EN-SkillsNetwork/labs/v1/m3/data/Project_data.csv

2. InvoiceNo starting with the character C is a credit entry and should be removed from the record before analysis.

3. StockCode values of C2, D, M, and POST correspond to Carraige, Discount, Manual and Postage entries, all of which are not required for our analysis.

4. There are a few entries where the CustomerID is missing. Such entries can be removed from the data before your analysis.

5. Load the final transaction record to an SQLite3 database Invoice_Records under the table Purchase_transactions.

Create a prompt and use it on the generative AI platform to generate a Python code that can create the required data processing pipeline.

Prompt:

For the transactional data table, create a python code that does the following.

1. Read the CSV data from a URL using Pandas

2. Remove entries where

a. InvoiceNo starts with the character 'C'

b. StockCode is either 'M', 'D', 'C2' or 'POST'

c. CustomerID is missing.

Print the length of the dataset before and after removal.

3. Load the final data to an SQLIte3 database named 'Invoice_Records' as a table 'Purchase_transactions'.

4. Run a sample query to display the first 5 rows of the table.

sample response:

```
import pandas as pd

import sqlite3

# Read the CSV data from a URL

url = "insert_your_csv_url_here"

data = pd.read_csv(url)



# Print the length of the dataset before removal

print("Length of dataset before removal:", len(data))

# Remove entries that meet the specified conditions
```

```
initial_length = len(data)

data = data[~data['InvoiceNo'].str.startswith('C')]

data = data[~data['StockCode'].isin(['M', 'D', 'C2', 'POST'])]

data = data.dropna(subset=['CustomerID'])

# Print the length of the dataset after removal

print("Length of dataset after removal:", len(data))

print("Number of entries removed:", initial_length - len(data))

# Load the final data to an SQLite3 database

conn = sqlite3.connect('Invoice_Records.db')

data.to_sql('Purchase_transactions', conn, if_exists='replace', index=False)

# Run a sample query to display the first 5 rows of the table

query_result = pd.read_sql_query("SELECT * FROM Purchase_transactions LIMIT 5;",
conn)

print(query_result)

conn.close()
```

Adjust Sample link:

```
import pandas as pd

import sqlite3

# Read the CSV data from a URL

url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMSkillsNetwork-AI0273EN-SkillsNetwork/labs/v1/m3/data/Project_data.csv"

data = pd.read_csv(url)

# Print the length of the dataset before removal

print("Length of dataset before removal:", len(data))

# Remove entries that meet the specified conditions

initial_length = len(data)

data = data[~data['InvoiceNo'].str.startswith('C')]

data = data[~data['StockCode'].isin(['M', 'D', 'C2', 'POST'])]

data = data.dropna(subset=['CustomerID'])

```

```
# Print the length of the dataset after removal

print("Length of dataset after removal:", len(data))

print("Number of entries removed:", initial_length - len(data))

# Load the final data to an SQLite3 database

conn = sqlite3.connect('Invoice_Records.db')

data.to_sql('Purchase_transactions', conn, if_exists='replace', index=False)

# Run a sample query to display the first 5 rows of the table

query_result = pd.read_sql_query("SELECT * FROM Purchase_transactions LIMIT 5;",
    conn)

print(query_result)

conn.close()
```

sample output:

```
theia@theia-abhishekg1:/home/project$ python3 test_file.py
Length of dataset before removal: 24180
Length of dataset after removal: 22420
Number of entries removed: 1760
  InvoiceNo StockCode                      Description  ...  UnitPrice CustomerID  Country
0    536370     22728        ALARM CLOCK BAKELIKE PINK  ...       3.75    12583.0   France
1    536370     22727        ALARM CLOCK BAKELIKE RED   ...       3.75    12583.0   France
2    536370     22726       ALARM CLOCK BAKELIKE GREEN  ...       3.75    12583.0   France
3    536370     21724  PANDA AND BUNNIES STICKER SHEET  ...       0.85    12583.0   France
4    536370     21883                 STARS GIFT TAPE   ...       0.65    12583.0   France

[5 rows x 8 columns]
```

Watson answer:

Here is the Python code that accomplishes the task:

python

*1*import pandas as pd

*2*import sqlite3

3

4# Read the CSV data from a URL

*5*url = 'https://cf-courses-data.s3.amazonaws.com/IBMSkillsNetwork-AI0273EN-SkillsNetwork/labs/v1/m3/data/Project_data.csv'

*6*df = pd.read_csv(url)

7

8# Remove entries where

9# a. InvoiceNo starts with the character 'C'

```
df = df[df['InvoiceNo'].str.startswith('C')].dropna()

# Remove entries where
# b. StockCode is either 'M', 'D', 'C2' or 'POST'
df = df[~df['StockCode'].isin(['M', 'D', 'C2', 'POST'])]

# Remove entries where
# c. CustomerID is missing
df = df.dropna(subset=['CustomerID'])

# Print the length of the dataset before and after removal
print("Before removal:", len(df))
print("After removal:", len(df))

# Load the final data to an SQLite database named 'Invoice_Records' as a table 'Purchase_transactions'
conn = sqlite3.connect('Invoice_Records.db')
df.to_sql('Purchase_transactions', conn, if_exists='replace', index=False)
```

28# Run a sample query to display the first 5 rows of the table

*29*print("Sample query:")

*30*print(df.head(5))

Here's a brief explanation of the code:

1. We first read the CSV data from the specified URL using pd.read_csv().
2. We then filter the data using boolean indexing to remove entries that meet the specified conditions:
    - df['InvoiceNo'].str.startswith('C') removes rows where the InvoiceNo starts with the character 'C'.
    - df[~df['StockCode'].isin(['M', 'D', 'C2', 'POST'])] removes rows where the StockCode is either 'M', 'D', 'C2', or 'POST'.
    - df.dropna(subset=['CustomerID']) removes rows where the CustomerID is missing.
3. We print the length of the dataset before and after removal using len().
4. We connect to an SQLite database named 'Invoice_Records' using sqlite3.connect().
5. We use to_sql() to load the filtered data into the database as a table named 'Purchase_transactions'.
6. Finally, we run a sample query to display the first 5 rows of the table using head().

Task 5 Querying the database

After the data is available on the central data repository, you can use SQL queries to extract the data directly into your Python coding interface. For the next part of your project, you are required to extract the data of a specific country–let's use Germany. Use the generative AI model to create a code snippet that you can add to your previous code in the testing interface and that will run a SQL query on the data to extract the details transactions for the specified country.

Prompt:

> Create a python code that will extract all records from the table 'Purchase_transactions' under the database 'Invoice_records', such that the Country parameter is set to Germany. The resulting record should be saved in a dataframe.

Sample response:

To extract all records from the 'Purchase_transactions' table under the 'Invoice_records' database where the Country parameter is set to Germany and save the resulting records in a dataframe in Python, you can use the following code snippet:

```
1. import sqlite3
2. import pandas as pd
3.
4. # Connect to the database
5. conn = sqlite3.connect('Invoice_records.db')
6.
```

```
# Execute a query to select all records from the 'Purchase_transactions' table with Country set to Germany
query = "SELECT * FROM Purchase_transactions WHERE Country = 'Germany'"
df = pd.read_sql(query, conn)

# Print the dataframe
print(df)

# Close the connection
conn.close()
```

This code snippet specifically filters the records where the Country is set to Germany and saves the results in a pandas dataframe df. Ensure you have the pandas library installed in your Python environment to run this code successfully.

Sample code for testing

```
import pandas as pd
import sqlite3

# Read the CSV data from a URL
```

```
5. url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMSkillsNetwork-AI0273EN-SkillsNetwork/labs/v1/m3/data/Project_data.csv"
6. data = pd.read_csv(url)
7.
8. # Print the length of the dataset before removal
9. print("Length of dataset before removal:", len(data))
10.
11. # Remove entries that meet the specified conditions
12. initial_length = len(data)
13. data = data[~data['InvoiceNo'].str.startswith('C')]
14. data = data[~data['StockCode'].isin(['M', 'D', 'C2', 'POST'])]
15. data = data.dropna(subset=['CustomerID'])
16.
17. # Print the length of the dataset after removal
18. print("Length of dataset after removal:", len(data))
19. print("Number of entries removed:", initial_length - len(data))
20.
21. # Load the final data to an SQLite3 database
```

```
conn = sqlite3.connect('Invoice_Records.db')

data.to_sql('Purchase_transactions', conn, if_exists='replace', index=False)

# Run a sample query to display the first 5 rows of the table
query_result = pd.read_sql_query("SELECT * FROM Purchase_transactions LIMIT 5;",
    conn)
print(query_result)

query = "SELECT * FROM Purchase_transactions WHERE Country IN ('Germany')"
records = pd.read_sql(query, conn)

# Print the extracted records
print(records)

conn.close()
```

sample output

```
theia@theia-abhishekg1:/home/project$ python3 test_file.py
Length of dataset before removal: 24180
Length of dataset after removal: 22420
Number of entries removed: 1760
  InvoiceNo StockCode                      Description  Quantity       InvoiceDate  UnitPrice  CustomerID Country
0    536370     22728        ALARM CLOCK BAKELIKE PINK        24  01-12-2010 08:45       3.75     12583.0  France
1    536370     22727        ALARM CLOCK BAKELIKE RED         24  01-12-2010 08:45       3.75     12583.0  France
2    536370     22726       ALARM CLOCK BAKELIKE GREEN        12  01-12-2010 08:45       3.75     12583.0  France
3    536370     21724  PANDA AND BUNNIES STICKER SHEET        12  01-12-2010 08:45       0.85     12583.0  France
4    536370     21883                 STARS GIFT TAPE         24  01-12-2010 08:45       0.65     12583.0  France
     InvoiceNo StockCode                          Description  Quantity       InvoiceDate  UnitPrice  CustomerID  Country
0       536527     22809              SET OF 6 T-LIGHTS SANTA         6  01-12-2010 13:04       2.95     12662.0  Germany
1       536527     84347  ROTATING SILVER ANGELS T-LIGHT HLDR         6  01-12-2010 13:04       2.55     12662.0  Germany
2       536527     84945   MULTI COLOUR SILVER T-LIGHT HOLDER        12  01-12-2010 13:04       0.85     12662.0  Germany
3       536527     22242        5 HOOK HANGER MAGIC TOADSTOOL        12  01-12-2010 13:04       1.65     12662.0  Germany
4       536527     22244           3 HOOK HANGER MAGIC GARDEN        12  01-12-2010 13:04       1.95     12662.0  Germany
...        ...       ...                                  ...       ...                ...        ...         ...      ...
8654    581578     22993         SET OF 4 PANTRY JELLY MOULDS        12  09-12-2011 12:16       1.25     12713.0  Germany
8655    581578     22907     PACK OF 20 NAPKINS PANTRY DESIGN        12  09-12-2011 12:16       0.85     12713.0  Germany
8656    581578     22908        PACK OF 20 NAPKINS RED APPLES        12  09-12-2011 12:16       0.85     12713.0  Germany
8657    581578     23215     JINGLE BELL HEART ANTIQUE SILVER        12  09-12-2011 12:16       2.08     12713.0  Germany
8658    581578     22736          RIBBON REEL MAKING SNOWMEN         10  09-12-2011 12:16       1.65     12713.0  Germany

[8659 rows x 8 columns]
```

Task 6 Data analysis and data mining

One of the most relevant and important techniques for analyzing transactional data is association rule mining. You are required to implement an Apriori algorithm to mine association rules from the data extracted in the previous step. As a result, you will be able to identify the items that have the most likelihood to be purchased together. This information is necessary for the company to efficiently develop their marketing and advertising strategies.

You can use generative AI to create the code for implementation. You can write a prompt that creates code for the implementation of Apriori algorithm for association rule mining on the extracted data frame.

Consider the following prompt that describes the different steps involved in implementing an Apriori algorithm.

Prompt:

1. For the data frame extracted here, write a python code to execute the apriori algorithm and extract association rules for the given set of invoices. For this outcome, perform the following steps:

    1. Group the records by InvoiceID and Description, along with their total quantities

    2. Unpack the data into a table, making the InvoiceNo, the row indexes, Unique descriptions as column indexes and the total quantities, the values.

    3. Apply one-hot encoding on this table, making the value True if the Item description existed in the invoice and False if it didn't exist in the invoice.

    4. Perfrom Apriori algorithm on this data and extract the required association rules.

sample response:

To execute the Apriori algorithm and extract association rules for the set of invoices extracted from the 'Purchase_transactions' table under the 'Invoice_records' database, you can follow these steps:

1. Group the records by InvoiceID and Description, along with their total quantities:

```
1. import sqlite3
1. import pandas as pd
2.
3. # Connect to the database
4. conn = sqlite3.connect('Invoice_records.db')
5.
6. # Execute a query to select all records from the 'Purchase_transactions' table
7. query = "SELECT InvoiceID, Description, SUM(Quantity) AS TotalQuantity FROM
   Purchase_transactions GROUP BY InvoiceID, Description"
8. df_grouped = pd.read_sql_query(query, conn)
9.
10. # Close the connection
11. conn.close()
```

2. Unpack the data into a table with the InvoiceNo” items as row indexes, Unique descriptions as column indexes, and total quantities as values:

   df_pivot = df_grouped.pivot(index='InvoiceNo', columns='Description', values='TotalQuantity').fillna(0)

3. Apply one-hot encoding on this table:

   df_encoded = df_pivot.applymap(lambda x: 1 if x > 0 else 0

4. Perform the Apriori algorithm using this data and extract the association rules:

```
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

# Apply the Apriori algorithm to find frequent itemsets
frequent_itemsets = apriori(df_encoded, min_support=0.1, use_colnames=True)

# Generate association rules from the frequent itemsets
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

# Print the association rules
print(rules)
```

You can comment out all previous print statements and combine these code snippets with the existing test code file, making completed code for reference.

Click here for sample final code

```
import pandas as pd

import sqlite3

from mlxtend.frequent_patterns import apriori

from mlxtend.frequent_patterns import association_rules

# Read the CSV data from a URL

url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMSkillsNetwork-AI0273EN-SkillsNetwork/labs/v1/m3/data/Project_data.csv"

data = pd.read_csv(url)

# Print the length of the dataset before removal

#print("Length of data set before removal:", len(data))

# Remove entries that meet the specified conditions

initial_length = len(data)
```

```
data = data[~data['InvoiceNo'].str.startswith('C')]

data = data[~data['StockCode'].isin(['M', 'D', 'C2', 'POST'])]

data = data.dropna(subset=['CustomerID'])

# Print the length of the data set after removal

#print("Length of data set after removal:", len(data))

#print("Number of entries removed:", initial_length - len(data))

# Load the final data to an SQLite3 database

conn = sqlite3.connect('Invoice_Records.db')

data.to_sql('Purchase_transactions', conn, if_exists='replace', index=False)

# Run a sample query to display the first 5 rows of the table

query_result = pd.read_sql_query("SELECT * FROM Purchase_transactions LIMIT 5;",
    conn)

#print(query_result)

query = "SELECT * FROM Purchase_transactions WHERE Country IN ('Germany')"

records = pd.read_sql(query, conn)
```

```
# Print the extracted records
#print(records)

# Execute a query to select all records from the 'Purchase_transactions' table
query = "SELECT InvoiceNo, Description, SUM(Quantity) AS TotalQuantity FROM Purchase_transactions GROUP BY InvoiceNo, Description"
df_grouped = pd.read_sql(query, conn)

df_pivot = df_grouped.pivot(index='InvoiceNo', columns='Description', values='TotalQuantity').fillna(0)

df_encoded = df_pivot.applymap(lambda x: 1 if x > 0 else 0)

# Apply the Apriori algorithm to find frequent itemsets
frequent_itemsets = apriori(df_encoded, min_support=0.05, use_colnames=True)

# Generate association rules from the frequent itemsets
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
```

```
50.

51. # Sort the association rules in descending order of confidence

52. rules = rules.sort_values(by='confidence', ascending=False)

53.

54. # Print the association rules

55. print(rules[['antecedents','consequents','confidence']])

56.

57. conn.close()
```

Click here for sample output

```
theia@theia-abhishekg1:/home/project$ python3 test_file.py
/home/project/test_file.py:43: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  df_encoded = df_pivot.applymap(lambda x: 1 if x > 0 else 0)
/home/theia/.local/lib/python3.10/site-packages/mlxtend/frequent_patterns/fpcommon.py:109: DeprecationWarning: DataFrames with non-bool typ
es result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
                                   antecedents                          consequents  confidence
7  (SET/6 RED SPOTTY PAPER CUPS, SET/20 RED RETRO...        (SET/6 RED SPOTTY PAPER PLATES)    0.964912
6  (SET/6 RED SPOTTY PAPER PLATES, SET/20 RED RET...          (SET/6 RED SPOTTY PAPER CUPS)    0.916667
2                    (SET/6 RED SPOTTY PAPER PLATES)          (SET/6 RED SPOTTY PAPER CUPS)    0.904762
3                      (SET/6 RED SPOTTY PAPER CUPS)        (SET/6 RED SPOTTY PAPER PLATES)    0.883721
4  (PLASTERS IN TIN SPACEBOY, PLASTERS IN TIN CIR...     (PLASTERS IN TIN WOODLAND ANIMALS)    0.764706
0                (ROUND SNACK BOXES SET OF 4 FRUITS )  (ROUND SNACK BOXES SET OF4 WOODLAND )    0.750000
5  (SET/6 RED SPOTTY PAPER PLATES, SET/6 RED SPOT...  (SET/20 RED RETROSPOT PAPER NAPKINS )    0.723684
1                    (SET/6 RED SPOTTY PAPER PLATES)  (SET/20 RED RETROSPOT PAPER NAPKINS )    0.714286
```

You can infer that if an item from the antecedent column is picked, then it can be said with the shown value of confidence that the corresponding consequent is also going to be picked for purchase in the same invoice.

Conclusion: **By applying generative AI in each step of the project, all tasks were completed in 15 minutes.**

Reference

IBM, Skill network, Rav Ahuja & Abhishek Gagneja.